

E.A.S. Protocol: Sistema interactivo para el análisis de redes mediante algoritmos de grafos y criptografía clásica

Aimer Esteban Garcia Rojas, Jair Gomez Narvaez, Diego Alejandro Robayo Cordoba
Programa de Ingeniería de Sistemas
Universidad Nacional de Colombia
Julio 2026

Abstract—Discrete mathematics provides the theoretical foundation for numerous applications in computer science. This paper presents the design and implementation of an interactive web application that integrates graph algorithms and classical cryptographic techniques into a single educational platform. The system enables users to create, edit, and visualize weighted graphs dynamically, as well as execute an extended implementation of Dijkstra's shortest-path algorithm that considers edge weights, communication risk, and node reliability. The reliability of each node can be modified through classical cryptographic techniques, including the Caesar, Hill and Affine ciphers, allowing users to analyze how these changes influence the computed shortest path. This approach provides an interactive environment for exploring and understanding key concepts of discrete mathematics through graphical visualization and experimentation. The application follows a modular client-server architecture, with a Python backend responsible for graph management and algorithm execution, and a JavaScript frontend that provides real-time interaction and visualization. Experimental evaluation using simulated network scenarios demonstrates that the platform correctly performs graph editing, shortest-path computation, and cryptographic operations while offering an intuitive environment for studying the practical application of discrete mathematics. The resulting system serves as an educational tool that combines algorithm visualization with interactive experimentation, facilitating both learning and future extensions.

Keywords—Graph, Dijkstra Algorithm, Classical Cryptography, Interactive Visualization.

I. INTRODUCCIÓN

Las matemáticas discretas constituyen uno de los pilares fundamentales de las ciencias de la computación y de múltiples ramas de la ingeniería y las matemáticas. Entre estos se encuentran los conjuntos, la lógica, las relaciones, conteo, combinatoria, entre otros[1]. En particular, la teoría de grafos proporciona modelos para representar redes de comunicación y resolver problemas de enrutamiento [1], mientras que la teoría de números sustenta numerosos algoritmos criptográficos utilizados para proteger la información [2]. Debido a ello, comprender estas estructuras resulta indispensable para el diseño y análisis de soluciones computacionales.

Sin embargo, muchos de estos temas suelen abordarse de manera independiente y mediante ejemplos exclusivamente teóricos, lo que dificulta apreciar la relación que existe entre ellos y su aplicación en problemas reales. En particular, la teoría de grafos permite modelar redes de comunicación y

encontrar rutas eficientes entre diferentes puntos, mientras que la criptografía clásica proporciona mecanismos para proteger la información que circula por dichas redes. En la práctica, ambos conceptos se complementan constantemente, ya que una comunicación eficiente también debe considerar aspectos relacionados con la seguridad y la confiabilidad de la información.

Con el propósito de integrar estos conceptos en un mismo entorno, este proyecto desarrolla un simulador interactivo de comunicación segura basado en grafos. La aplicación permite crear, modificar y visualizar dinámicamente una red de comunicación representada mediante un grafo ponderado dirigido, ejecutar una versión extendida del algoritmo de Dijkstra que incorpora el peso de las aristas, el riesgo de comunicación y el nivel de confiabilidad de los nodos, y analizar visualmente la ruta óptima obtenida. De manera complementaria, el sistema incorpora implementaciones de los cifrados clásicos César, Afin y Hill, con el fin de mostrar cómo diferentes mecanismos criptográficos pueden influir en las condiciones de seguridad de la red simulada.

El sistema fue desarrollado siguiendo una arquitectura cliente-servidor modular, utilizando Python para la implementación de los algoritmos y la lógica del sistema, y JavaScript para la construcción de una interfaz gráfica interactiva. Esta arquitectura facilita la separación de responsabilidades entre los diferentes componentes del proyecto y permite extender la plataforma con nuevos algoritmos o funcionalidades en futuras versiones, pensando en un ambiente de acceso por internet para mayor facilidad.

Como resultado, se obtuvo una herramienta educativa que integra algoritmos de teoría de grafos y criptografía clásica dentro de un mismo entorno visual e interactivo. Además de permitir la experimentación con diferentes configuraciones de la red, la plataforma facilita la comprensión de conceptos fundamentales de las matemáticas discretas mediante la observación directa del comportamiento de los algoritmos y del efecto que tienen las decisiones de seguridad sobre las rutas de comunicación.

II. FUNDAMENTOS DE MATEMÁTICAS DISCRETAS.

II-A. Teoría de grafos.

Un grafo es una estructura discreta formada por un conjunto de vértices (o nodos) y un conjunto de aristas que representan las relaciones existentes entre dichos vértices [2]. La teoría de grafos constituye uno de los pilares de las matemáticas discretas y de las ciencias de la computación, debido a su capacidad para modelar sistemas formados por entidades y relaciones entre ellas. Los grafos permiten representar redes de transporte, sistemas eléctricos, redes sociales, redes de computadores y redes de comunicación, entre muchas otras aplicaciones. Esta representación facilita la aplicación de algoritmos de optimización y análisis para resolver problemas de conectividad, búsqueda de caminos y planificación de rutas [3].

En este trabajo, la red de comunicación se modela mediante un grafo ponderado dirigido, donde cada vértice representa un dispositivo de la red y cada arista representa un canal de comunicación entre dos dispositivos, de la cual solamente se puede transmitir en la dirección a la cual esta apunta. Ya que no todos los canales de comunicación son iguales, puesto que unos pueden ser más eficientes que otros, a estas aristas se le asigna un peso [4] que determina la dificultad y/o eficiencia al recorrerlo. Con esto, al grafo se le denota como un grafo ponderado. A diferencia del algoritmo clásico de camino mínimo, cada nodo incorpora un nivel de confianza y cada enlace un nivel de riesgo, parámetros que influyen directamente en el costo utilizado durante la búsqueda de rutas.

Así, el tipo de grafo que va a protagonizar el simulador es aquel que se representa mediante la formula

$$G = (V, E, P)$$

donde V son los dispositivos en la red, E representa los canales de comunicación y w asigna un costo a cada enlace

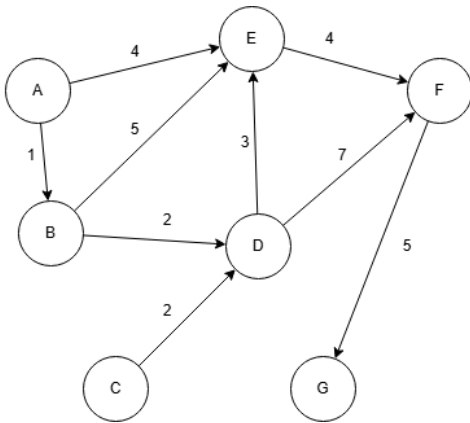


Figura 1. Grafo dirigido ponderado, donde los pesos representan el costo asociado a cada enlace de comunicación.

II-B. Algoritmos sobre grafos: Dijkstra y penalización

Uno de los problemas más estudiados en teoría de grafos es el problema del camino mínimo (Shortest Path Problem),

el cual consiste en determinar la ruta de menor costo entre un nodo origen y un nodo destino dentro de un grafo ponderado.

Entre los algoritmos propuestos para resolver este problema, el algoritmo de Dijkstra es uno de los más utilizados cuando los pesos de las aristas son no negativos. Su funcionamiento consiste en expandir progresivamente los nodos con menor costo acumulado conocido, actualizando las distancias de sus vecinos mediante un proceso conocido como relajación de aristas. Al finalizar la ejecución, el algoritmo garantiza encontrar el camino de costo mínimo desde el nodo origen hacia todos los nodos alcanzables del grafo[5].

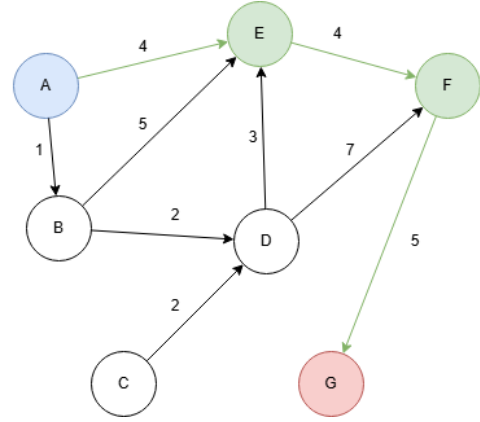


Figura 2. Camino más corto entre A y G usando Dijkstra implementado en un grafo ponderado

En este proyecto no se emplea el costo clásico basado únicamente en el peso de las aristas. Con el fin de representar aspectos relacionados con la seguridad de una red de comunicación, el costo de cada enlace incorpora dos factores adicionales: el nivel de riesgo asociado a la comunicación y el nivel de confianza del dispositivo destino. Cada costo se entiende expande con,

$$C_{ij} = P_{ij} + \lambda_c(1 - C_j) + \lambda_r R_{ij}$$

donde,

- C_{ij} : costo total de recorrer la arista
- P_{ij} : peso o distancia de la arista
- λ_c : factor de penalización por confianza
- C_j : nivel de confianza del nodo destino
- λ_r : factor de penalización por riesgo
- R_{ij} : riesgo de la arista

Como consecuencia, una ruta con menor distancia puede dejar de ser la mejor alternativa si atraviesa nodos con baja confianza o enlaces con un riesgo elevado. En contraste, una ruta ligeramente más larga puede resultar óptima al ofrecer un mayor nivel de seguridad. Para esta situación, Dijkstra ahora busca el costo mínimo, el cual es

$$C_{ruta} = \min \sum (P_i \lambda_c (1 - C_i) + \lambda_r R_i)$$

II-C. Teoría de números aplicada a la criptografía

La criptografía se creó con el objetivo de transformar un mensaje legible en otro que únicamente pueda ser interpretado

por quienes conocen la información necesaria para revertir el proceso. [6] Hoy en día, la criptografía constituye una de las principales aplicaciones de las matemáticas discretas dentro de la seguridad informática.[7] Aunque actualmente existen algoritmos criptográficos modernos de alta complejidad, los cifrados clásicos continúan siendo ampliamente utilizados con fines educativos debido a que permiten comprender conceptos fundamentales como la aritmética modular, el máximo común divisor, los inversos modulares y las transformaciones matriciales. Entre los métodos de encriptación clásica utilizados, en general se implementaron tres.

II-C1. Cifrado César y Afín: El cifrado César desplaza cada letra un número fijo de posiciones dentro del alfabeto. El cifrado Afín generaliza esta idea mediante una transformación afín sobre el conjunto de residuos módulo 26, definida por

$$E(x) = (ax + b) \pmod{26}$$

donde,

x : posición de la letra

a, b : claves

a es coprimo con 26

De esta forma se cifra el mensaje para cada letra. En cuanto al descifrado, se busca invertir las operaciones realizadas durante el cifrado. Como la transformación original multiplica por a y posteriormente suma b , primero es necesario eliminar el desplazamiento restando b , y luego deshacer la multiplicación utilizando el inverso modular de a . Así, la ecuación es

$$D(y) = a^{-1}(y - b) \pmod{26}$$

donde,

y : letra cifrada

a^{-1} : inverso modular de a

Para garantizar que el texto pueda recuperarse correctamente, el parámetro a debe poseer inverso multiplicativo módulo 26. Esto ocurre únicamente cuando a y 26 son coprimos, condición que se verifica mediante el algoritmo de Euclides para calcular el máximo común divisor.

II-C2. Cifrado Hill: Este algoritmo cifra grupos completos de letras simultáneamente mediante la multiplicación de un vector de texto por una matriz clave. Gracias a ello, la sustitución de cada símbolo depende de los demás caracteres del bloque, aumentando considerablemente la complejidad del análisis criptográfico.

En este método, cada letra del alfabeto se representa mediante un número entero entre 0 y 25. Posteriormente, el mensaje se divide en bloques de longitud n , donde n corresponde al tamaño de la matriz clave.

Así, sea P el vector correspondiente a un bloque del mensaje y K la matriz clave dimensión $n \times n$. El proceso de cifrado se define como

$$C = KP \pmod{26}$$

Para la implementación desarrollada se utiliza una matriz cuadrada de tamaño 3×3 , permitiendo cifrar tres caracteres en

cada iteración del algoritmo. Antes de comenzar el proceso, el texto se convierte a mayúsculas, se eliminan los caracteres que no pertenecen al alfabeto y, cuando es necesario, se completa el último bloque agregando la letra X hasta alcanzar el tamaño requerido.

Una vez validada la matriz clave, el descifrado requiere calcular su inversa módulo 26. Para ello se emplea la identidad matricial

$$K^{-1} = \det(K)^{-1} \text{adj}(K) \pmod{26}$$

Para el descifrado, al obtener la matriz inversa, cada bloque cifrado se transforma nuevamente mediante

$$P = K^{-1}C \pmod{26}$$

En conjunto, los conceptos anteriores constituyen la base matemática de la aplicación desarrollada. La teoría de grafos permite modelar la red de comunicación, el algoritmo de Dijkstra determina las rutas óptimas considerando múltiples criterios de costo, mientras que la teoría de números y el álgebra lineal sustentan los algoritmos criptográficos implementados. La integración de estos elementos permite representar, analizar y modificar dinámicamente el comportamiento de una red de comunicación segura dentro de un entorno interactivo.

III. DISEÑO DE LA SOLUCIÓN.

El sistema fue diseñado siguiendo una arquitectura cliente-servidor, con el objetivo de separar la lógica de negocio de la interfaz gráfica y facilitar futuras ampliaciones del proyecto. La aplicación se compone de un cliente web encargado de la interacción con el usuario y de un servidor que administra el estado del grafo, ejecuta los algoritmos de búsqueda y procesa las operaciones criptográficas. La comunicación entre ambos componentes se realiza mediante solicitudes HTTP que intercambian información en formato JSON. Esta organización permite mantener una clara separación de responsabilidades. Mientras el cliente se ocupa de la representación visual del grafo y de capturar las acciones del usuario, el servidor concentra todas las operaciones relacionadas con la manipulación de la red y el procesamiento de los algoritmos implementados.

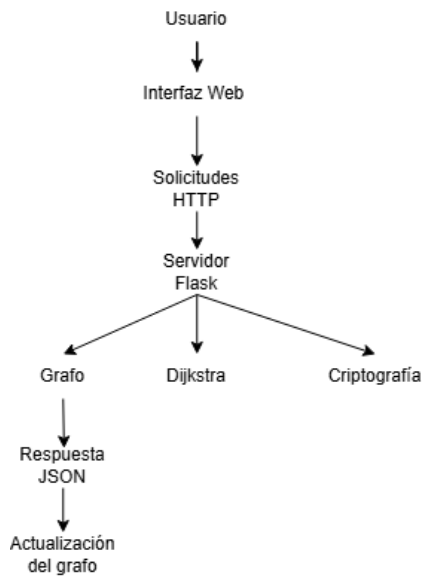


Figura 3. Arquitectura general del sistema

En la página principal se representa la red de comunicación mediante un grafo dirigido y ponderado. Cada dispositivo de red corresponde a un vértice, mientras que los canales de comunicación se representan mediante aristas dirigidas entre dichos vértices, como ya se ha explicado anteriormente. Como parte del diseño de interacción, se incorporó una barra de herramientas superior que agrupa las principales operaciones de modificación del grafo: creación de nodos, creación de aristas, eliminación de elementos y restauración del escenario inicial. Estas funcionalidades permiten al usuario construir y modificar dinámicamente la topología de la red antes de ejecutar los algoritmos de análisis.

Nodo: El modo de creación de nodos permite añadir nuevos dispositivos dentro de la red mediante una selección directa sobre el área de visualización del grafo. Para cada nodo generado, el sistema solicita un identificador único, una etiqueta descriptiva y un valor numérico correspondiente al nivel de confianza del dispositivo. Esta información es almacenada como parte de las características del vértice y posteriormente utilizada en el cálculo de rutas.

Arista: La creación de enlaces se realiza mediante la selección de un nodo origen y un nodo destino, estableciendo una relación dirigida entre ambos dispositivos. Cada conexión requiere la asignación de un peso asociado al costo de transmisión y un valor de riesgo que representa la posibilidad de una comunicación insegura. Estos parámetros intervienen posteriormente en la evaluación de las rutas calculadas por el algoritmo de Dijkstra modificado.

Eliminar: El sistema permite modificar la estructura de la red mediante la eliminación tanto de nodos como de aristas existentes. Cuando un nodo es eliminado, también se eliminan las conexiones asociadas, manteniendo la consistencia de la representación del grafo.

Reiniciar: La opción de reinicio permite recuperar la configuración inicial del escenario de prueba, descartando las

modificaciones realizadas durante la interacción. Esta funcionalidad facilita la repetición de experimentos bajo las mismas condiciones iniciales y permite evaluar nuevamente los algoritmos implementados.

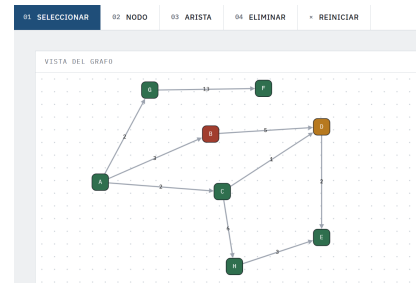


Figura 4. Grafo inicial con la barra de herramientas

Al no tener una herramienta de edición activa, el usuario puede interactuar libremente con la representación gráfica de la red, permitiendo desplazar los nodos para modificar su distribución visual. Además, mediante la selección de un vértice es posible consultar sus características asociadas en el panel de información ubicado en la parte derecha de la interfaz. Este panel presenta atributos como identificador, etiqueta, estado y nivel de confianza, permitiendo relacionar las propiedades de cada dispositivo con el comportamiento del algoritmo de búsqueda de rutas.

En la misma sección se encuentra el módulo de búsqueda de rutas seguras. Este componente recibe como entrada un nodo de origen, un nodo destino y los parámetros de ponderación relacionados con la confianza de los dispositivos y el riesgo de los enlaces. A partir de estos datos, el sistema ejecuta una versión modificada del algoritmo de Dijkstra, donde el costo de recorrido no depende únicamente del peso de la arista, sino también de penalizaciones asociadas a la seguridad de la comunicación. Como salida, el usuario obtiene la ruta seleccionada, el costo total calculado y un análisis general de las condiciones de seguridad de la trayectoria encontrada.

Finalmente, la plataforma incorpora un módulo de criptografía clásica orientado a la protección de mensajes dentro de la red simulada. Este módulo permite seleccionar diferentes métodos de cifrado, entre ellos los algoritmos Hill y Afín. Para el cifrado Hill, el usuario debe proporcionar valores para la matriz de clave de tamaño definido por el parámetro n , verificando que esta sea invertible bajo aritmética modular para garantizar el proceso de descifrado. En el caso del cifrado Afín, se ingresan los parámetros a y b , donde a debe cumplir la condición de ser coprimo con el módulo utilizado. Ambos algoritmos reciben un mensaje como entrada y generan como salida una versión cifrada del texto.

Las entradas y salidas del sistema se pueden resumir en las siguientes tablas

Cuadro I
ENTRADAS DEL SISTEMA

Entrada	Descripción
Creación de nodo	Identificador, nombre, confianza y posición
Creación de arista	Origen, destino, peso y riesgo
Consulta de ruta	Nodo origen y nodo destino
Cifrado	Texto y claves del algoritmo seleccionado

Cuadro II
SALIDAS DEL SISTEMA

Salida	Descripción
Grafo actualizado	Cambios realizados por el usuario
Ruta óptima	Camino encontrado por Dijkstra
Costo total	Valor acumulado de la ruta
Métricas	Peso, riesgo y penalización total
Texto cifrado	Resultado del algoritmo criptográfico
Texto descifrado	Recuperación del mensaje original

En genera, el flujo completo para hacer uso del sistema, con sus funcionalidades se presenta en la siguiente figura

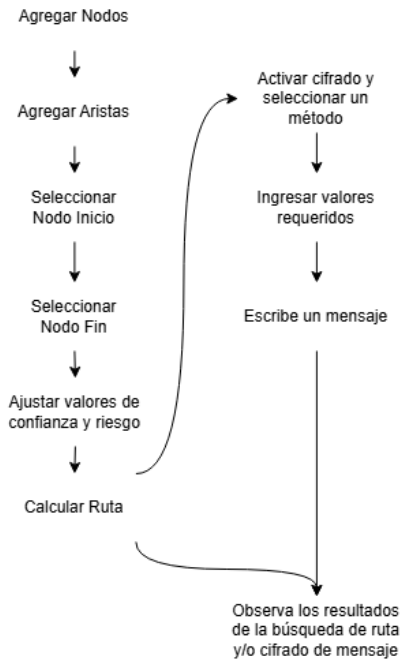


Figura 5. Flujo general de uso

IV. IMPLEMENTACIÓN

IV-A. Herramientas y tecnologías utilizadas

El sistema fue desarrollado siguiendo una arquitectura cliente-servidor, con una separación entre la lógica encargada del procesamiento de datos y la interfaz encargada de la interacción con el usuario. Para el desarrollo del servidor se utilizó Python debido a su facilidad para la implementación de estructuras discretas y algoritmos, además de la disponibilidad de librerías orientadas al manejo de datos y operaciones matemáticas. El backend fue construido utilizando el framework

Flask, encargado de administrar las solicitudes realizadas por el cliente, ejecutar los algoritmos definidos y retornar los resultados en formato JSON.

La interfaz gráfica fue desarrollada mediante tecnologías web como HTML, CSS y JavaScript. Para la representación interactiva de la red se utilizó la biblioteca Cytoscape.js, la cual permite visualizar grafos dinámicamente, modificar posiciones de los vértices y resaltar elementos específicos durante la ejecución de los algoritmos. Esta herramienta permitió relacionar directamente la representación matemática del grafo con una representación visual comprensible para el usuario.

IV-B. Arquitectura y estructura del sistema

La aplicación fue organizada mediante una arquitectura modular, donde cada componente tiene una responsabilidad específica dentro del funcionamiento general del sistema. Esta división permite facilitar el mantenimiento, la ampliación de funcionalidades y la integración de nuevos algoritmos.

El backend se estructuró en cuatro componentes principales:

IV-B1. Modelos: Contienen las estructuras utilizadas para representar los elementos necesarios para la construcción del grafo. Estos son nodo

Nodo
+ id: text
+ etiqueta: text
+ nivelConfianza: num
+ estado: text
+ x: num
+ y: num
+ dictar(dict): diccionario

Figura 6. Modelo de nodo

Nodo
+ inicio: text
+ fin: text
+ peso: num
+ riesgo: num
+ dictar(dict): diccionario

Figura 7. Modelo de arista

El modelo de grafo es más complejo, puesto que relaciona las clases de nodo y de arista con listas y diccionarios para permitir relacionarlos entre sí y formar una única estructura, con todos los métodos necesarios para trabajar sobre el grafo.

IV-B2. Algoritmos: Se encuentra el desarrollo e implementación de el algoritmo de búsqueda de caminos Dijkstra con las modificaciones necesarias para tener en cuenta el riesgo, confianza y penalización por seguridad. Se hizo uso de la librería heapq basada en una cola de prioridad.

IV-B3. Criptografía: Posee las dos implementaciones de criptografía clásica utilizada, las cuales son el cifrado Afín por generalización del cifrado de César y el cifrado de Hill. Cada uno posee sus propias confirmaciones de parametros y métodos de cifrado y descifrado. Se hizo uso de la librería numpy para las operaciones con matrices y vectores

IV-B4. Servicios: En este apartado se encuentran los servicios de simulador y cifrado. El simulador simplemente construye un grafo base de prueba con funciones sencillas para reiniciar y obtener un grafo. Por el otro lado, el cifrado toma el contenido de Criptografía y lo moldea para poder ser funcional interactivamente con el frontend.

IV-C. Comunicación entre componentes mediante rutas HTTP

La comunicación entre la interfaz gráfica y la lógica del sistema se realizó mediante rutas HTTP implementadas utilizando el framework Flask. Estas rutas funcionan como una capa intermedia entre el cliente y los algoritmos internos, permitiendo que las acciones realizadas por el usuario en la interfaz sean transformadas en operaciones sobre el modelo de datos.

Las solicitudes enviadas desde el cliente utilizan el formato JSON para transmitir información relacionada con los nodos, aristas, parámetros de búsqueda y mensajes a procesar. Posteriormente, el servidor interpreta estos datos, ejecuta la operación correspondiente y retorna una respuesta en formato JSON que permite actualizar dinámicamente la interfaz.

Las rutas implementadas se agrupan según la responsabilidad que cumplen dentro del sistema:

- **Gestión del grafo:** Incluye las operaciones relacionadas con la creación, consulta, modificación y eliminación de elementos de la red. Las rutas asociadas a los nodos permiten agregar nuevos dispositivos, obtener la información existente y eliminar elementos del modelo. De manera similar, las rutas de aristas permiten crear y eliminar conexiones entre dispositivos indicando sus atributos de peso y riesgo.
- **Inicialización y restauración del sistema:** La ruta de reinicio permite recuperar el grafo inicial de prueba definido dentro del simulador. Esta funcionalidad facilita realizar nuevas pruebas sin conservar modificaciones realizadas durante una ejecución anterior.
- **Ejecución de algoritmos:** La ruta asociada a Dijkstra recibe los nodos origen y destino junto con los parámetros de ponderación relacionados con confianza y riesgo. Esta información es enviada al algoritmo modificado de búsqueda de caminos, retornando la ruta encontrada, costos calculados, nodos visitados y métricas adicionales.
- **Servicios criptográficos:** Las rutas de cifrado permiten consultar los algoritmos disponibles y ejecutar procesos de cifrado o descifrado. Estas reciben el algoritmo seleccionado, el texto y la clave correspondiente, validando las restricciones matemáticas necesarias antes de realizar la operación.

IV-D. Estructura del cliente web

La interfaz del sistema fue desarrollada utilizando HTML, CSS y JavaScript, organizando la lógica del cliente mediante módulos independientes. Esta separación permite mantener

aisladas las responsabilidades relacionadas con la comunicación con el servidor, la manipulación del grafo y la interacción con el usuario.

Los principales componentes del cliente son:

- **Módulo de comunicación con el backend (api.js):** Este componente concentra las funciones encargadas de realizar solicitudes HTTP hacia las rutas Flask. A través de este módulo se gestionan operaciones como creación y eliminación de nodos, creación y eliminación de aristas, reinicio del grafo y ejecución de algoritmos de cifrado.
- **Módulo de manejo y actualización de la interfaz (interfaz.js):** Funciona como controlador principal de la interacción entre el usuario y las funcionalidades disponibles en la aplicación. Su responsabilidad es gestionar los eventos generados por los componentes HTML, actualizar los controles visuales y coordinar las solicitudes realizadas al servidor.
Este módulo administra los diferentes modos de operación del editor, permitiendo cambiar entre creación de nodos, creación de aristas, eliminación y selección de elementos. Actúa como intermediario entre las acciones realizadas por el usuario y los servicios disponibles en el backend, transformando las respuestas obtenidas en elementos visuales comprensibles.
- **Módulo de estado del editor (editor.js):** Mantiene la información temporal relacionada con la interacción del usuario, como el modo seleccionado actualmente, el nodo seleccionado, el nodo inicial utilizado para crear una arista y la instancia activa del grafo visualizado.
- **Módulo de visualización del grafo (grafo.js)** Contiene la lógica encargada de la representación y manipulación visual de la red mediante la biblioteca Cytoscape.js. Su función principal es transformar la información del grafo recibida desde el servidor en elementos gráficos compuestos por nodos y aristas. Este también implementa la edición visual del grafo y modifica dinámicamente los estilos visuales en el proceso de cálculo de rutas.
- **Módulo principal de inicialización (app.js):** Funciona como punto de entrada de la aplicación cliente y tiene como responsabilidad integrar los diferentes módulos desarrollados en JavaScript. Su objetivo es realizar la carga inicial de información, establecer la representación gráfica del sistema y preparar los componentes necesarios para la interacción del usuario.

REFERENCIAS

- [1] J. Tenjo, *Notas de clase: Matemáticas Discretas I*, Universidad Nacional de Colombia, 2026.
- [2] L. Lovász, J. Pelikán, K. Vesztergombi *Discrete Mathematics: Elementary and Beyond*. Springer. 2000. pp. 126
- [3] J. Villalpando y A. García, *Matemáticas Discretas, aplicaciones y ejercicios*. Patria. 2014. pp. 186
- [4] J. Villalpando y A. García, *Matemáticas Discretas, aplicaciones y ejercicios*. Patria. 2014. pp. 212
- [5] J. Villalpando y A. García, *Matemáticas Discretas, aplicaciones y ejercicios*. Patria. 2014. pp. 224-225
- [6] L. Lovász, J. Pelikán, K. Vesztergombi *Discrete Mathematics: Elementary and Beyond*. Springer. 2000. pp. 242

- [7] L. Lovász, J. Pelikán, K. Vesztegombi *Discrete Mathematics: Elementary and Beyond*. Springer. 2000. pp. v